

MBR Performance

The complete user guide to all-in-one WordPress speed optimisation — full page caching, core controls, scripts, styles, fonts, images, real-user monitoring and database housekeeping, explained one setting at a time.

Version 2.1.5

For WordPress 5.9 – 7.0 · Requires PHP 7.4 or newer

Compatible with classic and Full Site Editing (block) themes, all major page builders, multisite networks and WooCommerce.

NOTE

New in this version

2.1.5 removes a "strip_tags(): Passing null" deprecation notice that appeared at the top of the settings screen on PHP 8.1 and later with debugging switched on, and gives the screen a proper browser-tab title. **2.1.4** fixes Used CSS Mode B showing a static front page as "Still learning" forever: a template with only one URL can only ever give one sample, so the front page, cart and checkout now need just that one (section 6). Neither release changes how your site is optimised.

Earlier: 2.0.0 added the full page cache (section 21); 2.1.0 added an OPcache panel to the Diagnostics tab (section 13); 2.1.3 made Minify CSS and Minify JavaScript minify your files (sections 5 and 6).

Contents

- | | | | |
|----|----------------------------------|----|---|
| 1 | Getting Started | 12 | Server |
| 2 | How the Settings Work | 13 | Diagnostics |
| 3 | The Performance Doctor | 14 | Real User Monitoring |
| 4 | Core Features | 15 | Orphaned Media |
| 5 | JavaScript | 16 | WooCommerce |
| 6 | CSS UPDATED | 17 | Multisite Networks |
| 7 | Fonts | 18 | Recommended Workflow |
| 8 | Preloading & Speculative Loading | 19 | Troubleshooting & FAQ UPDATED |
| 9 | Lazy Loading | 20 | Glossary |
| 10 | Database | 21 | Full Page Cache |
| 11 | WebP / AVIF Images | | |

Every tab in this guide follows the same shape: a short explanation of what the tab is for, a description of each setting group, and clear guidance on what to switch on, what to leave alone, and what to test.

1 Getting Started

MBR Performance is an all-in-one optimisation plugin that gives you complete, transparent control over how WordPress builds and delivers your pages. Rather than a single "make it fast" button, it groups dozens of individual optimisations into clearly-labelled tabs so you can apply exactly what your site needs and nothing it doesn't.

Design principles

- **Caching is optional, not assumed** — the plugin has a full page cache of its own (section 21), switched off until you ask for it. If you already run a caching plugin it steps aside and leaves page caching off, and everything else carries on as before.
- **Safe by default** — almost every feature ships switched off. Nothing changes on your site until you deliberately enable it, so an install or update never alters how your pages render until you choose to.
- **One setting at a time** — features are deliberately granular. You enable one, test the front end, and move on, so if something does change unexpectedly you always know which switch caused it.
- **Self-contained** — no external CDN dependency for the plugin's own assets, and no telemetry. The only outbound connections are optional and clearly documented (Google Fonts, YouTube and Vimeo previews — see the relevant tabs).

Installing the plugin

- In WordPress admin, go to Plugins → Add New → Upload Plugin, choose the ZIP, then click Install Now.
- Alternatively, unzip it and upload the mbr-performance folder to /wp-content/plugins/ via SFTP or your host's file manager.
- Click Activate.
- The plugin appears as **MBR Performance** in the WordPress admin toolbar at the very top of the screen. Hover it to jump straight to any tab.

Deactivating and deleting

The two are deliberately different, and the difference matters if you are only switching the plugin off to test something.

- **Deactivating** is reversible and keeps your settings. It clears the plugin's scheduled jobs, removes its .htaccess blocks, removes the page-cache drop-in and empties its caches, so your site returns to how it behaved before — but every setting you configured is still there when you switch it back on.
- **Deleting** from Plugins → Delete removes the plugin's settings, its database tables and its caches as well. Your Media Library is never touched, and the WebP and AVIF files it generated are deliberately left in place, since a cached page or CDN may still be referring to them.

TIP**Before you change anything**

Take a full backup of your site (files and database). Then, if you possibly can, try new settings on a staging copy first. Enable one feature, save, and check the front end of your site in a fresh browser tab before moving to the next.

Requirement	Detail
WordPress	5.9 or newer (tested up to 7.0). Works on classic and Full Site Editing block themes. Script Modules support activates automatically on 6.5+.
PHP	7.4 or newer. AVIF image conversion additionally needs PHP 8.1+ with GD AVIF support, or Imagick 7.0.25+.
Server	Apache, LiteSpeed or Nginx. Browser-cache and compression rules are written to .htaccess on Apache/LiteSpeed; an equivalent snippet is shown for Nginx.
Compatibility	Elementor, Beaver Builder, Divi, Oxygen, Bricks and WPBakery. Optimisations are automatically suspended inside editor and preview modes.

2 How the Settings Work

All of the plugin's controls live on one settings screen split into a row of tabs. The order below is the order the tabs appear on screen, and the order they appear in the toolbar menu.

Tab	Purpose
Doctor	Analyses a real page and tells you which settings will help — the best place to start.
Cache	Full page caching: serving tiers, lifetime, exclusions, purging and debug headers (section 21).
Core Features	WordPress-level switches: emojis, embeds, REST API, feeds, HTML minify and more.
JavaScript	Defer, delay, move-to-footer, minify, combine, jQuery handling, script modules.
CSS	Used CSS Mode A and Mode B, async CSS, minify, combine, block styles, Critical CSS.
Fonts	Self-host Google Fonts, preload, subset, font-display, preconnect.
Preloading	Preload critical images, fetch priority, Early Hints, speculative loading.
Lazy Loading	Native image lazy-load, iframe/video lazy-load, exclusion rules.
Database	Revisions, drafts, trash, spam, transients, table optimisation, scheduling.
WebP / AVIF	Next-gen image conversion, bulk converters, image sizing & dimensions.
Server	Browser-cache headers and text compression via .htaccess.
Diagnostics	Autoloaded-options audit, WP-Cron viewer, conflict detector, OPcache status.
RUM	Real-user Core Web Vitals collected from your visitors and stored locally.
Orphaned Media	Find and safely remove unused Media Library files.
WooCommerce	Store-specific optimisations (only active when WooCommerce is installed).

Saving, resetting and tooltips

- Each tab has its own **Save Changes** button. Saving applies the settings on that tab immediately.
- Saving or resetting any tab also clears the plugin's generated files — combined and minified stylesheets and scripts, and the Used CSS caches — and the page cache, so every page is rebuilt with the new settings on its next visit. Nothing needs purging by hand after a change.
- Most settings carry a small ? tooltip. Hover it for a one-line explanation and any compatibility warnings specific to that option.

- A **Reset** control returns settings to their safe defaults if you need a clean slate.
- Tools that act on files or the database (bulk image conversion, database cleanup, orphaned-media deletion) have their own action buttons and progress bars, and run when you click them — not on save.

NOTE**The golden rule**

Change one thing, save, then load the front end of your site in a clean, logged-out browser tab (some optimisations are skipped for logged-in administrators). If everything looks right, move on; if not, switch that one setting back. A problem is never more than one toggle away from a fix.

3 The Performance Doctor

The Doctor is the recommended starting point. Instead of presenting a wall of switches, it loads a real front-end page from your site, measures what is actually slowing it down, and recommends — in priority order — which settings will help. Crucially, it also tells you which settings to leave off, and it skips anything you have already enabled.

What it checks

- **Render-blocking split.** The single most decisive factor: is your first paint being held up mainly by CSS or by JavaScript? The Doctor diagnoses the balance and points you to the CSS tab or the JavaScript tab accordingly, rather than letting you guess.
- **Image issues.** It flags images missing width/height attributes (which cause layout shift), JPEG/PNG images that a next-gen format would shrink, and below-the-fold images that aren't being lazy-loaded — each routed to the exact setting that fixes it.
- **Script modules.** It counts the ES modules a page loads, reports where the import map and each preload hint landed, and tells you whether preload hoisting would help here.
- **CSS delivery mode.** It knows which of the two Used CSS modes you are running, recommends one or the other but never both, and — when Mode B is on — tells you whether it is actually working on the page scanned.
- **Multiple templates.** It auto-detects your key page types — home page, blog index, a single post, a page, and (if present) the WooCommerce shop and a product page — then aggregates the findings into site-wide recommendations versus page-specific ones.
- **Real-user field data.** If Real User Monitoring is switched on (section 14), the Doctor leads with what your visitors actually experienced rather than what a synthetic scan predicts — including INP, which a page scan cannot measure at all.

Field data and synthetic analysis

A page scan is a laboratory measurement: it renders one page, once, from your server, and reasons about what it finds. That is genuinely useful, but it is not what your visitors experience on a mid-range phone on mobile data. When RUM has collected enough samples, the Doctor puts the field numbers first, names the element or handler most often responsible, and routes you to the setting that addresses it. Where the two disagree, field data wins — the Doctor will say so plainly. And when field data is still thin, it shows the provisional readings and tells you so, rather than drawing conclusions from a handful of samples.

Script modules and the import map

Script modules never enter the classic script queue, so the render-blocking pass steps over them by design — they are deferred by specification and cannot hold up first paint. The Doctor reports the module layer alongside everything else: its summary card shows how many modules the page loads, how many preload hints reached the head, and whether the import map was printed in the head or the footer. Module counts are deliberately kept out of the render-blocking JavaScript figure, because counting something that never blocks rendering would overstate the very problem the rest of the report is describing.

What it finds	What it tells you
Hoisting off, modules present	Recommends enabling preload hoisting, and says how many hints are currently stranded in the footer. Classic themes only.
Hoisting on, no hints yet	The URL hasn't been learned yet. The scan itself counts as the teaching visit, so re-running the Doctor should show the hints.
Hoisting on and working	Confirms how many hints are being emitted in the head. Nothing to do.
Block theme	Tells you plainly that WordPress already prints hints in the head here, and that there is nothing to change.
Module in head, map in footer	A high-priority warning — see below. This one is a breakage, not a slow page.

WARNING**When a module runs before the import map**

An import map has to be parsed before any module that depends on it. If a theme or plugin prints its own module tag directly into the page head while WordPress prints the map in the footer — which is where a classic theme puts it — then every bare specifier import in that module fails to resolve. Anything importing `@wordpress/interactivity` simply will not run.

This is a genuine fault rather than a missed optimisation, and no plugin setting can fix it: the module tag needs correcting at source, in whichever theme or plugin printed it. The Doctor raises it as a high-priority finding and points you at the browser console, where you will see the matching resolution errors.

What it says about your CSS

The CSS advice depends on which delivery mode you are running, because the two Used CSS modes are alternatives and enabling both would be worse than enabling either. With neither on, a CSS-heavy page gets a recommendation to enable Mode A, mentioning Combine CSS as the lighter-touch option and Mode B as the aggressive one. With Mode B on, the question changes: the Doctor reports whether the page still has render-blocking stylesheets, and if it does, tells you whether that template is simply still learning or whether the remaining sheets are ones Mode B leaves alone on purpose. See section 6.

Using the Doctor

- Open the Doctor tab and run a scan. A first-run nudge will steer you here automatically the first time you open the plugin; it dismisses itself once you've run a scan.
- **Scan key templates** checks your main page types at once and separates site-wide fixes from page-specific ones.
- Each recommendation links straight to the relevant setting, so you can act on it without hunting through tabs.
- When you're done, you can generate a branded, print-ready PDF report of the scan to hand to a client. The report is produced in your browser, so it adds no weight to the plugin and works on any host.

TIP**Advisory only**

The Doctor never changes any settings by itself. It only recommends. You stay in full control of what actually gets switched on — run a scan, read the advice, then decide. Re-run it after making changes to confirm the improvement.

NOTE**Where the real win is**

If the Doctor reports that your render-blocking is mostly CSS, the fix is on the CSS tab (Used CSS / async). If it's mostly JavaScript, the fix is on the JavaScript tab (Defer / Delay). Spending effort on the wrong one is the most common way to get a disappointing result, which is exactly what the Doctor is there to prevent.

4 Core Features

The Core tab switches off WordPress features and front-end clutter that most sites never use. None of these are dramatic speed wins on their own, but together they trim requests, remove inline scripts and reduce your site's surface area. The settings are grouped into sections.

Scripts & Styles

Setting	What it does	When to use
Disable Emojis	Removes the emoji-detection script and inline styles	Safe on almost any site that doesn't rely on legacy emoji rendering
Disable Dashicons	Stops the Dashicons icon font loading on the front end for logged-out visitors	Only when no front-end feature needs it
Disable Embeds	Removes oEmbed, so YouTube/Twitter URLs no longer auto-embed	When you paste media as blocks rather than bare URLs
Disable jQuery Migrate	Drops the compatibility shim for old jQuery code	Only if no plugin/theme needs it — test thoroughly
Remove Global Styles	Strips the inline global-styles CSS block	Classic themes only — auto-skipped on block themes
Conditional Block Styles	Loads block CSS only when a block is actually present	Generally safe and worthwhile

WARNING Remove Global Styles and block themes

On a Full Site Editing (block) theme — Twenty Twenty-Two onwards and similar — the inline global-styles block is what renders your colours, fonts, spacing and layout on the public front end. The plugin automatically skips this optimisation when a block theme is active, but the safest approach is to leave it off unless you run a classic theme.

WordPress Features

- **Disable XML-RPC** — closes a legacy remote-access endpoint. Improves security but can break the Jetpack connection and some mobile apps.
- **Remove WordPress version** — hides the version number from the page head and feeds (a small security-hardening measure).
- **Disable RSD / shortlinks / self-pingbacks** — removes rarely-needed discovery and ping features from the head.
- **Disable RSS feeds, or just hide feed links** — turn feeds off entirely, or keep them working but remove their discovery links from the head.

- **REST API control** — choose between leaving the REST API fully open, disabling it for non-admins, or disabling it when logged out. An allowlist field lets you keep specific namespaces public (for example a contact form, store API or chat widget) even when the API is otherwise restricted.
- **Disable AI Features (WordPress 7.0+)** — WordPress 7.0 ships a built-in AI Client, Abilities API and a Connectors screen. They stay dormant until you connect a provider, but this toggle switches the whole subsystem off using core's own kill switch. It has no effect on WordPress 6.x, so it's safe to leave on across mixed-version sites.

TIP**REST API: don't go all-or-nothing**

If you need to lock the REST API down, use the allowlist rather than a blanket block. Common public namespaces to keep open include contact-form and store endpoints. Blocking everything is the usual cause of "the contact form / store suddenly stopped working" after hardening. The plugin's own `mbrpe/v1` namespace is always permitted, so features like Real User Monitoring keep working however you set this.

Advanced Performance — Minify HTML

Also on the Core tab, **Minify HTML** strips comments and collapses excess whitespace from your page output. It is carefully built to preserve the contents of script, style, pre, textarea and inline SVG blocks, to honour IE conditional comments, and to skip pages that embed a complete nested HTML document (such as a full landing page inside a page-builder HTML widget). AMP, REST, AJAX, feed, embed and customiser-preview responses are never touched, and if anything fails to round-trip the original output is restored wholesale. Off by default.

NOTE**Modest by design**

Once gzip or brotli compression is active (see the Server tab), HTML minification saves relatively little. Treat it as a small finishing touch rather than a headline optimisation, and leave it off if you ever see odd whitespace-sensitive layout. It is separate from Minify CSS and Minify JavaScript, which shrink your stylesheet and script files (sections 5 and 6).

5 JavaScript

JavaScript is the most common cause of a slow or janky first load, so this tab is where many of the biggest wins live — but it's also where careless changes break sites. Enable one option at a time and test interactive elements (sliders, menus, forms, cart) after each.

Loading strategy

Setting	What it does	When to use
Defer JavaScript	Lets the page render first, then runs scripts in order	A strong, generally-safe first step; each script honours its own exclusion list
Move scripts to footer	Relocates scripts below the page content	When a theme/plugin enqueues scripts in the head unnecessarily
Delay JavaScript	Holds non-critical scripts until first user interaction	Excellent for analytics and third-party widgets; set a timeout fallback
Async JavaScript	Runs scripts as soon as they load, out of order	For independent scripts that don't depend on others

TIP**Delay is your friend for third-party scripts**

Chat widgets, analytics and tag managers rarely need to run before the visitor interacts. Delaying them until the first scroll, tap or move can transform a PageSpeed score with no visible downside. Keep a timeout so they still fire on pages where the visitor never interacts.

jQuery handling

- **Defer jQuery** — defers the core jQuery library along with everything else. Test first, as some inline scripts expect jQuery to be ready immediately.
- **Remove jQuery** — for modern sites that don't depend on it. A built-in test mode lets you check whether anything breaks before committing.

NOTE**jQuery is protected from the footer move**

"Move scripts to footer" never relocates the jQuery foundation (jquery, jquery-core and jquery-migrate). Moving those splits the dependency graph — jquery-migrate could be left in the head while jquery-core dropped to the footer, producing "jQuery is not defined" and silently breaking page-builder widgets such as Elementor accordions. jQuery always stays in the head, and the exclusion list continues to apply to everything else. Advanced users can adjust the protected set with the `mbrpe_footer_protected_handles` filter.

Minify JavaScript

Minify JavaScript serves a minified copy of each local script that is not already minified — the files PageSpeed Insights lists under "Minify JavaScript". Comments and whitespace are removed with a proper JavaScript tokeniser (the bundled JShrink library) rather than pattern matching, so strings, regular expressions and template literals come through exactly as written.

The first request for each script writes its copy to `/uploads/mbr-performance-min/`, in a folder that mirrors the script's own path, and every later page links to the copy. The copy's filename carries a fingerprint of the original, so when a plugin or theme update changes a script a fresh copy is made automatically. The script's handle, load order, defer and delay handling are unchanged — only the URL it loads from.

Inline script attached to enqueued scripts — the configuration blocks WordPress and plugins print beside them — is minified in place.

Left exactly as it is	Why
Files named .min.js	Already minified by their author.
Files a copy would barely shrink	If minifying saves under 5% the file was already minified; the decision is remembered for that version of the file.
Scripts that find their own URL	Webpack bundles, workers and anything using <code>document.currentScript</code> load further files from beside themselves, and would look in the wrong folder.
External scripts	Served from another domain, such as a CDN or Google, so there is no local file to minify.
Anything that fails to parse	Served unchanged — a script is never delivered half-processed.
Your exclusion list	Handles or URL fragments in "Exclude from Minification/Combination". Because the copy mirrors the original's path, fragments such as <code>plugins/elementor</code> still match it.

Combine JavaScript

Combine JavaScript merges multiple local scripts into one cached bundle to cut HTTP requests, processed separately for the head and footer groups. For safety it only combines "pure" scripts: any script carrying inline or localised data (which routinely includes per-request security nonces), an `async/defer` strategy, or a conditional, is left alone. Scripts in your Defer, Delay and Exclude lists are also respected. With Minify JavaScript on, each script in a bundle is minified first.

- **Remove script versions** — drops the `?ver=` query string for better edge caching.

NOTE

Why Combine JS merges fewer files than Combine CSS

This is expected, not a fault. CSS can be merged freely, but a script carrying inline data (often a security nonce) can't be safely baked into a shared cached file, so it breaks the run and stays separate. You still get the big win — jQuery and the cluster of vanilla libraries folded together — just with more standalone files than on the CSS side. If a combined script misbehaves (typically one that loads its own workers via its script URL), add it to the Exclude list.

The tab shows how many combined and minified files are currently cached, with a one-click **Clear JS file cache** button. Files rebuild automatically when your settings or scripts change, so the button is just a manual flush; using it also clears the page cache, so no stored page is left pointing at a removed file.

Script Modules & the Interactivity API

WordPress 6.5 introduced a second way of loading JavaScript. Alongside the traditional script queue, it can load code as ES modules — used by the Interactivity API and by interactive blocks. These are printed separately from ordinary scripts and are accompanied by an import map, a small piece of JSON that tells the browser how to resolve each module's imports.

This section appears only on WordPress 6.5 and newer. On older versions the API does not exist and the controls are hidden entirely.

NOTE

Defer, delay, combine and minify never touch modules — by design

Modules are deferred by specification, so deferring them again is meaningless. More importantly, folding them into a combined bundle would destroy the import map that resolves their imports, breaking the page. The plugin therefore leaves modules strictly alone, and there is no exclusion list to maintain — the classic optimisation passes structurally cannot reach them.

Module preload hoisting

This is the setting that earns its place, and it fixes a genuine gap — but only on classic themes. On a block theme, WordPress knows which modules a page needs while it is still building the head, so it prints its preload hints there — early, where hints are useful. On a classic theme (which includes most page-builder sites), modules are discovered while the page body renders, so WordPress has no choice but to print the import map and every preload hint in the footer. By then the hint arrives at the same moment as the script it was meant to fetch ahead of, which makes it close to worthless.

With hoisting enabled, the plugin learns which modules each URL actually loads, then emits the preload hints in the head on subsequent visits — so the browser starts fetching the module graph while it is still parsing the head.

Setting	What it does	When to use
Hoist module preloads	Emits learned modulepreload hints in the page head	Classic themes running interactive blocks; no effect on block themes
Maximum preloads	Caps how many hints are emitted per page	Keep modest — preloading everything competes with what matters
High priority modules	Marks named modules as high fetch priority	Requires WordPress 6.9+; ignored safely on earlier versions
Never preload	Leaves named modules out of the hints	For modules you'd rather the browser fetched only when needed

Only static imports are hinted. Dynamic imports — modules a page loads on demand, if at all — are deliberately left out, since preloading them would fetch bytes the visitor may never need.

TIP**The first visit teaches, the second benefits**

Hoisting works by learning. The first visit to a URL records which modules it loads; the hints appear from the second visit onwards. That also means you should press **Clear learned modules** after changing themes or plugins, so the hints are relearned from the current module set. Upgrading the plugin clears the learned set automatically for the same reason.

NOTE**Let the Doctor tell you whether this applies to you**

Rather than guessing whether your site uses modules at all, run a scan on the Doctor tab. It reports how many modules the page loads, where the import map and preload hints landed, and whether hoisting is off, still learning, or working — and it will tell you outright if you are on a block theme and there is nothing to gain here. See section 3.

6 CSS

If the Doctor reports that your render-blocking is mostly CSS, this is the tab that fixes it. There are two ways to do it — the safe one and the aggressive one — and they are alternatives, never a pair. The tab also minifies and combines your stylesheets, which is covered at the end of the section.

Two modes, one job

Both modes do the same thing in outline: work out which CSS rules are actually needed, inline those in the page head so the first paint has everything it requires, and stop the full stylesheets from blocking render. What separates them is what happens to the original stylesheets, and how widely each analysis is applied.

	Mode A	Mode B
Analysed per	Each URL	Each template
Original stylesheets	Deferred — still downloaded, just later	Removed — not downloaded at all
If a rule is missed	The full sheet arrives a moment later and corrects it	Nothing corrects it — the rule is gone for that template
Cache entries	One per page	One per template
Learns from	The page itself	Several URLs of the same template
Suits	Almost any site; the sensible default	Sites where you can test every template before going live

NOTE

Mode A or Mode B — never both

The two modes are alternatives and only one can own CSS delivery for a page. If you switch Mode B on, Mode A stands down automatically and the CSS tab says so next to its toggle — as do Async CSS and Combine CSS, for the same reason. The Doctor will recommend one or the other, never both.

Used CSS (Mode A)

With **Generate Used CSS** enabled, the plugin works out which CSS rules each page actually uses, inlines just those rules in the page head for an instant first paint, and loads your full stylesheets asynchronously behind them. This removes render-blocking unused CSS from the critical path.

It is the safe variant of unused-CSS removal: the original stylesheets are never deleted, only deferred. So if the analysis misses a rule — for example one used by a widget that JavaScript injects after load — the full stylesheet still arrives a moment later and corrects it. Used CSS is generated in the background after the first visit to each page and cached per URL, so the first visitor triggers generation and later visitors get the optimised version. With the plugin's own page cache on, generating renders are served but not stored, and the page's cache entry is cleared once its used CSS is ready, so the optimised page is what gets cached.

When Used CSS helps (and when it doesn't)

- It earns its place on CSS-heavy sites — a large theme plus a page builder plus several plugins each loading their own stylesheet.
- It only helps if your render-blocking is genuinely caused by CSS. Check PageSpeed Insights: if the render-blocking list is mostly stylesheets, Used CSS helps; if it's mostly JavaScript, the win is on the JavaScript tab instead.
- On a site already light on CSS, the gain may be small.

TIP**Getting the best from Used CSS**

Leave it off by default and test on a staging copy first. If a particular stylesheet must always load in full, add part of its URL to the **Exclude from optimization** field. There's no need to also enable Async CSS — Used CSS handles delivery itself and automatically suppresses the standalone async layer to avoid the two conflicting.

Used CSS Mode B (aggressive)

Mode B is the counterpart to Mode A, and it differs in three ways that all follow from one decision: it removes the stylesheets it has analysed rather than deferring them. The bytes genuinely stop being downloaded, which is the whole point — and because nothing arrives behind the inlined CSS to correct a mistake, everything else about the feature is built around not making one.

- **It analyses templates, not pages.** Front page, blog index, pages, single posts, archives, and — on a store — shop, product, cart and checkout. A site with ten thousand posts keeps one cache entry for "single posts", not ten thousand.
- **It learns each template from several URLs.** Because one page is a poor witness for a whole template, Mode B samples a few different URLs and keeps the sum of what they used. A rule only one of your posts needs still survives for all of them.
- **It only removes what it has seen.** A stylesheet is dropped from a page only if that exact file was analysed while learning. Everything else keeps loading exactly as it does today.

The learning phase

Mode B does nothing at all on the first few visits to each template — those requests are served completely untouched while the analysis runs afterwards, so a visitor never sees a half-optimised page. Once a template has collected the number of samples you set, every later visit to that template is served the inlined CSS with the analysed stylesheets removed. Visit your site logged out to teach it: logged-in views are excluded, because your admin bar is not what a visitor gets.

The template cache table on this tab shows how many samples each template has, how many stylesheets it replaces, the size of the inlined CSS, and whether it is learning or serving. A static front page, the cart and the checkout only ever have one URL, so each is learned from that single visit.

What Mode B will not remove

The list below is deliberately cautious. Leaving a stylesheet in place costs one request; removing one it should not have costs you an unstyled page, so every uncertain case resolves the same way.

- **Any sheet it did not analyse.** If a plugin loads a stylesheet on only some pages of a template, that sheet keeps its link on every page. This is the single rule that makes per-template removal safe.
- **Sheets containing @import.** An @import cannot survive being inlined, and with the original removed the imported file would never arrive at all. Such sheets are left alone entirely.

- **Print and narrow-media sheets.** A print sheet does not block render anyway. A sheet limited to a media query would need that query rebuilt around the extracted rules, and getting that subtly wrong is invisible on your own monitor. A media="screen" sheet is handled, its rules re-wrapped so they cannot leak into print.
- **External stylesheets**, the admin bar and Dashicons, and anything on either exclusion list.
- **Requests with query arguments** — a filtered archive or paged listing renders markup the samples may not cover, so those are served untouched. Campaign and click-ID parameters (utm_*, gclid, fbclid and friends) are the exception: they change nothing about the page, so ad and newsletter landings still get the optimisation.

Setting	What it does	Guidance
Enable Mode B	Turns on per-template analysis and stylesheet removal	Off by default. Stands Mode A down automatically
URLs sampled per template	How many distinct URLs teach each template	Three suits most sites. Higher is safer and slower to settle
Selector safelist	Class names or prefixes always kept	The guard rail — see below. End an entry with * to match a prefix
Never remove these stylesheets	URL fragments left completely alone	For a sheet you would rather load in full than reason about

TIP**The safelist is the setting that matters**

The analysis reads your page as the server delivers it, so anything JavaScript adds afterwards is invisible to it and its CSS looks unused: a consent banner that appears on scroll, a cart drawer, a modal, a lightbox, a sticky header class toggled at a scroll position. List those class names in the safelist, one per line, and end an entry with * to match a prefix. Common framework prefixes are already handled for you. If something looks wrong after enabling Mode B, this is almost always the fix.

WARNING**Test every template before going live**

Mode B removes stylesheets, so a rule it wrongly drops stays dropped for that whole template until you safelist it and clear the cache. Enable it on a staging copy first, then walk every template logged out — home, a post, a page, an archive, and your shop and product pages if you have them — and interact with everything: open the menu, the search, the cart, any modal or slider. Anything that only appears after a click is exactly what the analysis cannot see.

To compare a page against its original stylesheets at any time, add ?mbrpe_modeb=off to its URL. That single request is served completely untouched.

When the cache clears itself

A template that has removed stylesheets must never outlive the markup it was measured against, so Mode B clears its whole cache whenever content is saved, the theme is switched, a plugin is activated, deactivated or updated, the Customizer is saved, or you change any plugin setting. That is blunter than Mode A, deliberately: a stale Mode A page is slightly wrong, whereas a stale Mode B page is unstyled. Templates simply relearn on the next few visits. There is also a **Clear template cache** button for when you want to force it.

Minify CSS

Minify CSS serves a minified copy of each local stylesheet that is not already minified — the files PageSpeed Insights lists under "Minify CSS". Comments and unnecessary whitespace are removed; quoted strings, url() values and the spacing inside values such as calc() are preserved exactly.

Copies are written to /uploads/mbr-performance-min/ on first request, in folders that mirror each stylesheet's own path, and are rebuilt automatically when the original changes. Because a copy lives in a different folder, relative url() and @import references — fonts, background images — are rewritten to full URLs pointing back at the original files. Inline CSS attached to enqueued stylesheets, including WordPress's global styles, is minified in place.

Files named .min.css, files a copy would shrink by less than 5% (often the case once relative URLs are written out in full), external stylesheets, the admin bar and Dashicons, and anything on your exclusion list keep their original URL. Minify CSS works alongside either Used CSS mode: the inlined CSS is already minified, and the stylesheets Mode A defers are served as minified copies.

Other CSS settings

Setting	What it does	When to use
Minify CSS	Serves minified copies of local stylesheets and minifies inline CSS (above)	Generally safe; also minifies the combined bundle
Async CSS	Loads stylesheets without blocking render (preload + onload)	When you're not using either Used CSS mode; both supersede it
Combine CSS	Merges adjacent same-origin stylesheets into one bundle	Cuts requests; stood down automatically under either Used CSS mode
Preload Combined CSS	Emits an early preload hint for each bundle	Only with Combine CSS on; auto-skipped when Async CSS is on
Remove CSS versions	Drops the ?ver= query string from stylesheet URLs	Better edge caching
CSS scanner	Reports unused styles for manual review	Investigation aid, not an automatic change

NOTE

Used CSS or Combine CSS — not both

These are alternatives. Either Used CSS mode already owns CSS delivery, so when one is on, Combine CSS is automatically stood down. The Doctor will recommend one approach, never a stack of them. Minify CSS is different: it changes the size of each file rather than how files are delivered, so it combines safely with any of them.

The tab shows how many combined and minified stylesheets are cached, with a **Clear CSS file cache** button that also clears the page cache.

Critical CSS

Separately from both modes, the CSS tab accepts hand-pasted critical CSS produced by a viewport-aware external generator, stored in per-context slots. When a slot matches the page being rendered, that pasted CSS owns delivery for that page and everything else — including Mode B — stands down on it. Use this when you want exact control over a specific template and are willing to generate the CSS yourself.

7 Fonts

Web fonts are a frequent and easily-fixed source of slow first paint and privacy headaches. This tab lets you self-host Google Fonts, control how they load, and remove them entirely where they aren't needed.

Self-hosting Google Fonts

Enable **Self-host Google Fonts** and download a font from this tab. Your server fetches the stylesheet and font files from Google once, stores them locally, and serves them from your own domain thereafter — so your visitors' browsers never contact Google. This removes a third-party connection, speeds up font delivery, and helps with GDPR concerns around Google Fonts. Family names are case-sensitive ("IBM Plex Sans", not "Ibm Plex Sans"), and the weight must be one the family publishes.

NOTE**What gets sent to Google, and when**

The download happens only at the moment you trigger it in the admin area. Your server sends the requested font family name(s) and standard request information (its IP address and user agent) to Google's font servers. No website-visitor data is sent. Once downloaded, fonts are served entirely from your own server.

Loading and display controls

- **Preload critical fonts** — tells the browser to fetch your most important fonts early, reducing the flash of invisible/unstyled text.
- **Preconnect to font domains** — warms up the connection to a font host before it's needed (useful if you keep some fonts remote).
- **Font subsetting** — strips unused characters to shrink font files.
- **Font-display strategy** — choose swap, block, fallback or optional to control how text behaves while a font loads (swap is the usual choice for showing text immediately).
- **Font Awesome optimisation** — trims icon-font overhead where it applies.
- **Manual font management** — add and manage your own font files directly.

Removing Google Fonts entirely

Disable Google Fonts strips Google Fonts from your pages completely — both the googleapis.com and gstatic.com requests. It also catches Google Fonts that a theme hardcodes directly into the page (header link tags, preconnects, inline @font-face and @import) which bypass WordPress's normal enqueue system, via a guarded final-output pass. There's also a dedicated Elementor Google Fonts control and a Clear font cache button.

TIP**A practical font workflow**

For most sites: enable Self-host Google Fonts, download the families your theme actually uses, set font-display to swap, and preload only the one or two fonts visible above the fold. Then disable any remaining remote Google Fonts so nothing slips out to Google in the background.

8 Preloading & Speculative Loading

Where lazy loading defers what isn't needed yet, preloading does the opposite: it fetches the few resources the browser will need immediately, earlier than it otherwise would. Used sparingly, it improves perceived speed and your Largest Contentful Paint (LCP).

Preloading critical resources

- **Preload critical images** — fetch your LCP/hero image early so the most important visual appears sooner. Use this only for the genuine above-the-fold image; preloading everything defeats the purpose.
- **Fetch Priority** — automatically assigns high priority to the first image, lets you mark additional critical images via CSS selectors, and can disable WordPress core's own fetch-priority behaviour if it's conflicting.
- **Cloudflare Early Hints (HTTP 103)** — supports the Early Hints mechanism so a supporting host/CDN can start delivering resource hints before the full response is ready.

Speculative loading

Speculative loading makes the next navigation feel instant by fetching or even rendering the likely next page in advance.

Setting	What it does	When to use
Prefetch mode	Fetches the next page's resources ahead of the click	Low-risk, broad speed-up for navigation
Prerender mode	Fully renders the next page in the background	Strongest effect; use moderate eagerness to avoid waste
Eagerness level	Conservative, moderate or eager — how aggressively to speculate	Start conservative/moderate and increase if needed
Auto mode	Lets the browser decide the optimal strategy	A sensible hands-off default

NOTE**Don't over-preload**

Preloading and prerendering consume bandwidth and CPU. Reserve image preloading for your single hero image, and keep speculative eagerness moderate unless you've confirmed a clear benefit. Aggressive prerendering on a site with many outbound links can waste resources fetching pages nobody visits.

9 Lazy Loading

Lazy loading defers off-screen images, iframes and embedded videos until the visitor scrolls near them, so the initial page weighs far less and loads faster. The key to using it well is excluding the images that must appear immediately.

What you can lazy-load

- **Native image lazy loading** — uses the browser's built-in lazy-load for images.
- **iframes and embedded videos** — defers YouTube, Vimeo and other embeds until needed.
- **Video facade** — replaces a heavy embedded player with a lightweight thumbnail and play button; the real player loads only when the visitor clicks. This saves a large amount of JavaScript on first load and stops the embed setting cookies until interaction.

NOTE

How the video facade fetches previews

To show the placeholder thumbnail, the visitor's browser requests the preview image from YouTube (i.ytimg.com) or Vimeo's public API. That request includes the visitor's IP address and the video ID. The full player and any video-provider cookies load only if the visitor clicks play. The facade is keyboard-accessible (Enter/Space).

Excluding critical images

The most important setting here isn't what you lazy-load, but what you exclude. Your logo and above-the-fold hero image should load immediately — lazy-loading them hurts your LCP. The plugin lets you exclude images by:

- CSS selectors
- Class names and IDs
- Data attributes
- Keywords found in the image's src or class
- Parent element selectors

TIP

The rule for lazy loading

Lazy-load everything below the fold; never lazy-load what's above it. After enabling, scroll your home page and a couple of key templates: anything visible without scrolling — logo, hero, first product image — should appear instantly. If one fades in late, add it to an exclusion rule.

10 Database

Over time a WordPress database accumulates clutter — old revisions, spam, abandoned drafts, expired transients — that bloats tables and slows queries. This tab cleans it up, on demand or on a schedule.

What it cleans

- **Post revisions** — trim to a configurable number kept per post (uses the proper WordPress filter, so the limit actually takes effect).
- **Auto-drafts and trash** — delete old auto-saved drafts and emptied-trash items past a configurable age.
- **Spam and unapproved comments** — remove spam and pending comments older than a set age.
- **Orphaned metadata** — clear post, comment, term and relationship metadata left behind by deleted content.
- **Transients** — purge expired transients (handles network/site transients on multisite).

Table maintenance

- **Optimise tables** — reclaim space and defragment database tables.
- **Convert MyISAM to InnoDB** — move legacy tables to the modern storage engine for better concurrency and reliability.
- **Repair tables** — attempt repair on tables flagged as damaged.

These tools act only on the current site's own tables. On a multisite network, a Super Admin on the main site also reaches the network-shared tables.

Scheduling

Set the cleanup schedule to daily, weekly or manual. The scheduled job re-aligns itself automatically to whatever you choose, runs all the enabled cleanups together, and writes a **Last Auto-Cleanup** log to this tab so you can see exactly what was removed and when. A **Run Auto-Cleanup Now** button lets you trigger it on demand.

WARNING**Cleanup is permanent — back up first**

Deleting revisions, drafts, trash and comments cannot be undone from inside the plugin. Take a database backup before your first cleanup, and start with conservative retention values (for example keep more revisions and a longer trash age) until you're comfortable with what's being removed.

11 WebP / AVIF Images

Images are usually the heaviest thing on a page. This tab converts them to modern, far smaller formats and fixes the most common image-related PageSpeed warnings. It's one of the highest-impact tabs in the plugin.

Next-gen formats: WebP and AVIF

The plugin converts your JPG, JPEG and PNG images to WebP and, where your server supports it, AVIF — typically 20–30% smaller than WebP again at equivalent perceived quality. Converted images are delivered through an HTML `<picture>` wrapper that offers AVIF first, then WebP, then the original JPEG/PNG as a fallback, so each browser automatically picks the first format it supports. Your originals are never thrown away.

- **Convert on upload** — every new image is converted automatically as it's added.
- **Bulk converters** — convert your existing Media Library in batches, with a progress bar and per-image AJAX so large libraries don't time out. Separate WebP and AVIF runs.
- **Compression level** — set quality from 1–100 to balance size against fidelity.
- **Smart skip** — if a converted file would be larger than the original, it's skipped automatically.
- **Conversion history** — a table showing each image with its WebP and AVIF sizes and the saving achieved, plus a registry-driven **Revert All** that deletes only the files the plugin created, never your originals.
- **Server diagnostics** — confirms GD/Imagick support so the AVIF tools only appear when your host can actually encode AVIF.

NOTE

Apache / LiteSpeed rewrite option

On Apache or LiteSpeed you can optionally serve WebP transparently via .htaccess rewrite rules. On other servers the `<picture>` delivery handles format selection in the browser instead — either way, unsupported browsers always receive the original. The rules work wherever WordPress is installed, including a subdirectory.

Image Sizing & Dimensions

This section tackles two more PageSpeed warnings: "Properly size images" and "Ensure images have explicit width and height".

- **Resize on upload** — automatically scales oversized uploads down to a configurable maximum (default 2560px), preserving aspect ratio via the WordPress core pipeline.
- **Inject width & height** — adds missing dimension attributes to front-end images so the browser can reserve space, reducing Cumulative Layout Shift (CLS). Works on post content, Gutenberg blocks, Elementor widgets and thumbnails; external images, SVGs and data URIs are skipped.
- **Bulk resize** — scan for existing images above your maximum, then downscale them in place with a progress bar and live log; sub-sizes and stale WebP files are handled automatically.
- **decoding="async" and EXIF stripping** — let images decode off the main thread, and remove camera/GPS metadata from new JPEGs (ICC colour profiles are preserved).

WARNING**Bulk resize overwrites files permanently**

Unlike resize-on-upload, the bulk resize tool rewrites existing files on disk and cannot be undone automatically. Take a full backup before running it, and let the scan show you what will change before you start the resize.

12 Server

Two of the most common PageSpeed Insights warnings — "Serve static assets with an efficient cache policy" and "Enable text compression" — are fixed at the web-server level. This tab writes the necessary rules for you.

Browser-cache headers

Adds Expires and Cache-Control headers so returning visitors' browsers reuse your static assets instead of re-downloading them: roughly one year for images and fonts, thirty days for CSS and JavaScript. On Apache and LiteSpeed the rules are written to .htaccess inside a clearly-marked MBR Browser Cache block.

Text compression

Enables Brotli and Gzip compression for text resources (HTML, CSS, JavaScript, SVG and so on), typically cutting their transfer size by a large margin. Written to .htaccess in an MBR Compression block.

Compression and minification complement each other: minify first removes what the browser never needed, and compression then shrinks what is left.

Nginx hosts

Nginx doesn't use .htaccess. The plugin detects your web server and, on Nginx, shows you an equivalent configuration snippet to paste into your server block instead. Both marker blocks are removed cleanly if you deactivate the plugin, or if you untick both boxes and save.

NOTE**Check with your host's cache**

Many managed hosts already set cache headers and compression at the server or CDN level. If yours does, you may not need these toggles — and the Diagnostics tab's conflict detector will help you spot overlap with optimisation plugins you already run.

13 Diagnostics

The Diagnostics tab is a set of investigative tools that surface performance problems WordPress otherwise hides from you.

- **Autoloaded Options Audit** — autoloaded options are loaded on every single page request, so a bloated `wp_options` table is one of the most common silent performance killers. This tool shows your total autoloaded size and the 30 largest options, with a one-click **Disable autoload** button. Protected core options (`siteurl`, `home`, `active_plugins`, `template`, `stylesheet` and the like) cannot be touched, and transients are flagged.
- **WP-Cron Viewer** — lists every scheduled event with its next run, recurrence, and a column showing whether a PHP callback is actually registered for it. Events with no callback — left behind by deactivated plugins — are flagged as orphan and can be unscheduled in one click. It also explains how to replace WP-Cron with a real system cron for performance-sensitive sites.
- **Caching Plugin Conflict Detector** — detects popular caching/optimisation plugins (WP Rocket, W3 Total Cache, LiteSpeed Cache, FlyingPress, WP Super Cache, Perfmatters, Autoptimize and SiteGround Optimizer) and lists exactly which MBR options overlap with each, so you never run the same combine, minify or defer pass twice.

TIP**Run the conflict detector early**

If you use any caching or optimisation plugin alongside this one, check the conflict detector before enabling Combine, Minify or Defer here. Running the same pass in two plugins is the classic cause of broken layouts and scripts — pick one tool to own each job.

OPcache

OPcache keeps compiled PHP in memory so WordPress is not recompiled on every request. It is configured by your host in `php.ini`, so no plugin can switch it on or tune it; this panel reports what the server is doing and lets you flush it. On multisite it is shown to super admins only, because one OPcache serves every site on the server.

- **Status** — hit rate, memory in use, cached files against the server's limit, restart counts, whether PHP checks for changed files, and when OPcache last restarted.
- **File-cache mode** — some hosts, SiteGround among them, run OPcache with compiled PHP stored on disk rather than in shared memory. The panel recognises this and shows the mode and whether PHP checks for changed files; memory and hit-rate figures do not exist in this mode, and Flush OPcache is not offered because it would leave the disk cache untouched.
- **Plain-English findings** — flags OPcache being missing or off, restricted by the host, or running short of memory or file slots, naming the setting to ask your host about.
- **Flush OPcache** — clears compiled PHP so it is rebuilt from disk, also available from the admin toolbar. Use it after changing PHP files outside WordPress — SFTP, a file manager or a Git deploy.
- **Automatic refresh** — the files the plugin writes itself (`advanced-cache.php`, the cache configuration and `wp-config.php`) are refreshed the moment they change, so settings apply on the next request even where PHP never checks for changes. On WordPress older than 6.2, updated plugins, themes and translations are refreshed too; newer versions do this themselves.

14 Real User Monitoring

Every other measurement in this plugin is synthetic: the Doctor renders a page from your server and reasons about what it finds. That is useful, but it isn't what your visitors experience on a mid-range phone on patchy mobile data. Real User Monitoring collects Core Web Vitals from the people actually using your site, and stores every one of them on your own server.

NOTE**Nothing leaves your server**

RUM contacts no third party at all. The measurement library is bundled with the plugin and served from your own domain, and the readings are posted to an endpoint on your own site and stored in your own database. No cookies are set, no IP addresses are stored, and the user agent is reduced to a coarse device class and browser family before anything is written. The endpoint is rate-limited and capped on total stored rows.

What it measures

Metric	What it tells you	Good
LCP	Largest Contentful Paint — how long your biggest visible element takes to appear. The headline perceived-speed number.	under 2.5s
CLS	Cumulative Layout Shift — how much the page jumps around while loading. Usually images without dimensions, or a font swap.	under 0.1
INP	Interaction to Next Paint — how quickly the page responds when someone taps or clicks. Only exists once a real person interacts.	under 200ms

TIP**INP is the reason to switch this on**

LCP and CLS can be estimated by a page scan. INP cannot — it only comes into existence when a real visitor taps, clicks or types, so no synthetic tool can measure it, however sophisticated. If you have ever wondered why a page scores well in testing but feels sluggish in the hand, INP is usually the answer, and RUM is the only way to see it.

Reading the panel

- **Scorecard** — your p75 for each metric, with the good / needs-improvement / poor split shown as a bar. The p75 is the 75th percentile: the experience of your slower quarter of visits, which is the standard way Core Web Vitals are judged.
- **By template** — the same figures broken down by page type and by desktop versus mobile. This is where the panel earns its keep, because it tells you where the problem is rather than just that one exists.
- **Worst offenders** — the specific URLs with the most poor readings, and the element or event handler most often responsible. This turns "your INP is 380ms" into "your INP is 380ms, and it's the menu toggle".
- **Data health** — how many raw samples and aggregate rows are stored, with a control to clear the lot.

Settings

Setting	What it does	Guidance
Enable RUM	Starts collecting from real visitors	Off by default. Adds a small script to front-end pages
Metrics	Choose which of LCP, CLS and INP to collect	Leave all three on
Sample rate	Percentage of page views that report	100% is fine for most sites; lower it on very high traffic
Exclude logged-in users	Keeps your own admin visits out of the data	Leave on — your logged-in experience isn't typical
Retention	How long raw samples and daily aggregates are kept	Defaults of 3 and 60 days suit most sites

How the data is processed

Each reading arrives as a raw sample. A nightly job rolls those into daily per-template and per-URL figures, then deletes the raw rows beyond your retention window, so the tables never grow without bound. Opening the RUM tab also triggers that roll-up on demand, so you don't have to wait for the overnight run to see fresh numbers.

NOTE

Provisional readings

Below ten samples a metric is shown greyed and marked as provisional, and the Doctor will not raise a recommendation from it. A p75 drawn from four visits isn't a measurement, it's an anecdote. Let real traffic accumulate for a few days before acting on what you see — and remember that visits from a logged-out browser are what count, since logged-in sessions are excluded by default.

A sensible way to use it

- Switch RUM on and leave it alone for a week. There is nothing to tune while it collects.
- Come back and read the By template table first. A single slow template is a far more actionable finding than a site-wide average.
- Check the worst-offender element for your weakest metric, then act on the Doctor's recommendation for it — it will point you at the tab that fixes that specific cause.
- Re-check a week after making changes. Field data moves slowly because it reflects real traffic, so give it time before judging the result.

15 Orphaned Media

Over the life of a site, the Media Library fills with files that nothing references any more — replaced images, abandoned uploads, leftovers from deleted posts. This tab finds them and removes them through a deliberately cautious, reversible workflow.

How detection works

A scan checks each attachment against several sources of references: its post parent, use as a featured image, appearances in post content (matched by attachment ID, shortcode reference and filename stem, so sized variants and URL-only references are caught), and a string-search across all postmeta values. You can scan images only, or extend the scan to videos, audio, documents and archives via the tab's checkboxes.

The two-tier safety classifier

- **High-confidence orphans** (zero references found anywhere) are eligible for bulk deletion.
- **Review candidates** (matched only in postmeta) must be inspected and deleted individually — this tier deliberately covers references stored in page-builder data that the scan can't fully parse.

The deletion and restore workflow

- Before anything is deleted, a staging table records the full attachment record, its postmeta and a file manifest.
- You choose a restore window — 7, 14, 30 or 60 days, or "keep forever". A daily job purges staging records once their window expires.
- At delete time, orphan status is re-verified: if an attachment has become referenced since the scan, the deletion is blocked.
- For images, the original, all sub-size variants, the scaled full-size variant and matching .webp siblings are removed together; for other media, the single file is removed. Every file is confirmed to sit inside the uploads directory before it is removed.
- A per-attachment exclusions list permanently keeps chosen IDs off the orphan list.
- Scans run with a live progress bar, batched in chunks of 50 to avoid timeouts on large libraries.

WARNING**Restore brings back the record, not the file**

Restoring within the window reinstates the database record, but the image bytes are physically deleted at the time of staging — a restored attachment must be re-uploaded if you need the file itself. Always review the candidate list carefully (especially the review tier) before bulk-deleting, and back up first.

16 WooCommerce

This tab activates only when WooCommerce is installed, and consolidates the store-specific optimisations that make the biggest difference to a shop's speed. When WooCommerce isn't present, the tab shows an inactive state so the capability stays discoverable.

Key optimisations

- **Cart fragments control** — WooCommerce fires an admin-ajax request on every page load to refresh cart fragments, which is a major drag on cached sites. Disable it site-wide, or only on non-shop pages. This is often the single biggest WooCommerce TTFB win.
- **Conditional asset loading** — dequeues WooCommerce scripts, styles, block assets, selectWoo and blockUI on pages that aren't part of the store, so your blog and landing pages don't carry shop weight.
- **Disable the password strength meter** (zxcvbn) on the front end.
- **Disable marketplace suggestions and dashboard widgets**, and prevent the heavy wc-admin React bundles from loading on non-WooCommerce admin screens.
- **Action Scheduler retention** — cap how long completed scheduled actions are kept (default 30 days, with shorter options) to stop the actionscheduler_actions table ballooning on busy stores.
- **One-click cleanups** — clear expired WooCommerce sessions and product/order/expired transients, with an optional weekly automated cleanup and a last-run log.

NOTE

Geolocation vs. page caching

The tab warns you if WooCommerce's default customer location is set to "Geolocate", which breaks full-page caching, or "Geolocate with page cache support", which appends a query string some caches mishandle. A direct link takes you to the WooCommerce setting to resolve it.

TIP

Test the mini-cart after enabling cart fragments

If your theme relies on a live-updating mini-cart that changes the moment an item is added, disabling cart fragments site-wide can stop it updating without a page reload. If that matters to your store, restrict the setting to non-shop pages instead, and test the add-to-cart experience.

17 Multisite Networks

On a WordPress Multisite network the plugin can be managed centrally from the Network Admin, so you set your performance policy once and apply it everywhere.

- **Network activation** — activate (and deactivate) the plugin across the whole network at once.
- **Network defaults** — manage a single set of default settings from Network Admin → Settings → MBR Performance.
- **Push to all sites** — apply your network defaults to every site, or a selection of sites, in one click.
- **Import from a site** — take the settings from any existing site and adopt them as the network defaults.
- **Per-site override control** — super admins decide whether individual site admins may customise their own settings or are locked to the network defaults. When overrides are locked, the per-site Save and Reset controls are disabled and a notice explains why.
- **Automatic setup for new sites** — newly-created sites in the network inherit the network defaults automatically.

TIP**A sensible network policy**

Configure one representative site exactly how you want it, import those settings as your network defaults, then push to all sites. Decide deliberately whether to lock overrides: locking keeps the network consistent, unlocking lets site owners tune their own pages.

18 Recommended Workflow

Putting it all together, here is a safe order of operations that gets most sites most of the way with the least risk.

Step	What to do
1 Back up	Take a full backup, and work on a staging copy if you can.
2 Switch on RUM	Enable Real User Monitoring first and let it start collecting while you work through everything else. By the time you've finished, you'll have real field data to check your results against.
3 Run the Doctor	Let it tell you whether your render-blocking is CSS or JavaScript, which image issues to fix, and whether the script-module layer needs anything. Act on its priority list rather than guessing.
4 Fix images first	On the WebP / AVIF tab, bulk-convert to next-gen formats and inject missing width/height. Images are usually the biggest single win and the lowest risk.
5 Address render-blocking	If the Doctor flagged CSS, enable a Used CSS mode — Mode A unless you are able to test every template, in which case Mode B goes further. If it flagged JavaScript, enable Defer and Delay non-critical scripts. Do one, test, then the other.
6 Minify	Switch on Minify CSS and Minify JavaScript, one at a time, and click through your key templates. Low risk, and it clears the "Minify" items in PageSpeed Insights for files on your own domain.
7 Tidy fonts	Self-host Google Fonts, set font-display to swap, and preload only above-the-fold fonts.
8 Set lazy loading	Lazy-load below-the-fold images and videos; exclude your logo and hero image.
9 Server headers	Enable browser-cache headers and compression — unless your host or CDN already provides them (check Diagnostics).
10 Page cache	Switch on page caching (section 21) and confirm X-MBR-Cache reports HIT on a second visit.
11 Database housekeeping	Trim revisions, clear clutter, and set a sensible cleanup schedule.
12 Check for conflicts	Run the Diagnostics conflict detector if you use any other caching/optimisation plugin, and switch off any duplicated pass.
13 Re-measure	Re-run the Doctor and PageSpeed Insights, click through your key templates logged out, and — after a few days — check the RUM tab to confirm real visitors are seeing the improvement.

TIP**If you are choosing between the two Used CSS modes**

Start with Mode A. It is safe by construction, needs no learning period, and on most sites captures the majority of the available gain. Move to Mode B only when you have a staging copy, a finite set of templates, and the time to click through each one — what it buys you is the bytes themselves, not just their timing.

19 Troubleshooting & FAQ

Something on my site broke after I changed a setting. What now?

Because the plugin is granular and you changed one thing at a time, the cause is the last setting you enabled — switch it back off and the problem should clear. If you're unsure which it was, reset the relevant tab to defaults and re-enable features one at a time, testing the front end after each.

Can I use this alongside a caching plugin?

Yes. Everything except page caching works alongside caching plugins and host-level caches; MBR Performance's own page cache (section 21) stays off while another is active. Where a feature overlaps with something your cache plugin also does — combine, minify, defer and so on — run it in one place only. The Diagnostics conflict detector flags the exact overlapping toggles for you.

PageSpeed Insights still says "Minify CSS" or "Minify JavaScript".

Check the URLs it lists. Before version 2.1.3 these options did not minify files at all, so if you are on an older version, update. After that, three things account for anything still listed. Files served from another domain — a CDN, Google, a chat or analytics provider — are outside the plugin's reach. A file on your exclusion list is left alone by request. And if a page cache, a host cache or a CDN is serving a copy of the page stored before you switched the option on, it will still link to the original files: the plugin's own page cache is cleared when you save, but purge any other cache by hand, then re-test.

You can confirm the option is working in view-source: local stylesheet and script URLs should point into `/wp-content/uploads/mbr-performance-min/`.

Some of my files aren't being minified.

That is usually deliberate. Files already named `.min.css` or `.min.js`, files that would shrink by less than 5%, scripts that load further files from beside themselves (webpack bundles, workers), and scripts the tokeniser cannot parse all keep their original URL, because serving a copy would gain nothing or risk breaking them. Sections 5 and 6 list every case.

A script misbehaves only with Minify JavaScript on.

Add its handle, or part of its URL, to **Exclude from Minification/Combination** on the JavaScript tab and save. That script is then served from its original file. If you can, tell us which plugin or theme it came from, so the automatic checks can learn to recognise it.

Combine JavaScript merged far fewer files than Combine CSS. Is it broken?

No — that's correct behaviour. CSS can be merged freely, but a script carrying inline data (which often includes per-request security nonces) can't be safely baked into a shared cached file, so it's left on its own. You still get the main benefit of folding jQuery and the vanilla libraries together; you just keep more standalone files on the JS side.

Used CSS didn't speed up my site.

Used CSS only helps when your render-blocking is genuinely caused by CSS. Check PageSpeed Insights: if the render-blocking list is mostly JavaScript, the win is on the JavaScript tab (Defer/Delay) instead. On sites already light on CSS the gain is naturally small.

Should I use Used CSS Mode A or Mode B?

Mode A unless you have a reason to go further. It is safe by construction: the original stylesheets still load behind the inlined CSS, so a rule the analysis misses is corrected a moment later. Mode B removes those stylesheets, which is a real additional saving — the bytes never travel — but it means a missed rule stays missed for that template. Choose Mode B when you can test every template on staging first, and use the selector safelist for anything JavaScript adds after load.

I enabled Mode B but my pages look exactly the same.

Three things to check. First, Mode B learns before it acts: the first few visits to each template are served untouched while it samples them, and only once a template has enough samples does it start serving the optimised version. The template cache table on the CSS tab shows how far each has got. Second, logged-in views are excluded entirely — browse logged out or in a private window. Third, if a cache other than the plugin's own is serving a copy stored before you enabled the feature, clear it.

Something looks wrong on one template under Mode B.

Almost always the analysis could not see the element, because JavaScript adds it after the page loads — a cookie banner, a cart drawer, a modal, a class toggled at a scroll position. Add the relevant class name to the Selector safelist on the CSS tab and save; saving clears the template cache, so the next few visits relearn with that selector kept. To confirm the diagnosis before changing anything, load the page with `?mbrpe_modeb=off` appended — that single request is served with its original stylesheets, so if the problem disappears you have found the cause.

Mode B is on but the Doctor still reports render-blocking CSS.

Either that template has not finished learning — re-run the scan and the number should fall — or the remaining stylesheets are ones Mode B deliberately leaves alone: sheets it did not see while learning, sheets containing `@import`, print and narrow-media sheets, and anything on your exclusion lists. That is by design. Removing a stylesheet the analysis never examined is precisely the mistake that produces an unstyled page.

Fonts or background images went missing after an earlier update.

Versions before 2.1.3 had a fault in the CSS minifier that broke quoted `url()` values in combined bundles and in the CSS inlined by Used CSS and Mode B. Updating to 2.1.3 fixes it and clears the affected caches automatically, so everything is rebuilt correctly on the next visit to each page.

I've enabled RUM but the tab shows no data.

Three things to check. First, visits from a logged-in browser are excluded by default, so browse your site logged out or in a private window. Second, readings only appear once they've been rolled up — opening the RUM tab does this for you, so simply reloading it is usually enough. Third, below ten samples a metric is marked provisional rather than treated as a measurement. Give it real traffic for a few days.

Why does the Doctor say my field data is "provisional"?

Because there aren't yet enough samples for the number to mean anything. A p75 calculated from a handful of visits will swing wildly. The Doctor shows you the readings so you know collection is working, but deliberately won't raise a recommendation from them until there's enough data to trust.

Does RUM send anything to a third party?

No. The measurement library is bundled with the plugin and served from your own domain, and readings go to an endpoint on your own site. No cookies, no IP addresses, no external requests of any kind.

Do the defer, combine and minify settings affect script modules?

No, and they shouldn't. Modules used by the Interactivity API and interactive blocks are printed separately by WordPress and are deferred by specification already. Combining them would break the import map that resolves their imports, so the plugin leaves them strictly alone. The one module setting that does something is preload hoisting, and only on classic themes.

I enabled module preload hoisting but nothing appeared in the head.

Hoisting learns before it acts, so the first visit to a URL records its modules and the hints appear from the second visit onwards. It also does nothing on a block theme, where WordPress already prints the hints in the head correctly. Rather than guessing, run the Doctor on that page — it will tell you which of those two situations you're in, or confirm the hints are being emitted. If you've recently changed theme or plugins, press **Clear learned modules** so the set is relearned.

The Doctor says a module script runs before the import map. What does that mean?

An import map has to be parsed before any module that relies on it, and on a classic theme WordPress prints that map in the footer. If a theme or plugin prints its own module tag straight into the page head, it sits above the map — so any bare specifier it imports, such as `@wordpress/interactivity`, cannot be resolved and the module never runs. Check the browser console on that page and you'll see the resolution errors. No plugin setting can fix this: the module tag needs correcting in whichever theme or plugin printed it.

Does it work with my page builder?

Yes — Elementor, Beaver Builder, Divi, Oxygen, Bricks and WPBakery are all supported, and optimisations are automatically suspended inside editor and preview modes so the builder keeps working normally.

My logo or hero image fades in late.

That image is being lazy-loaded when it shouldn't be. On the Lazy Loading tab, add it to an exclusion rule (by class, ID, selector or a keyword in its filename) so it loads immediately. Never lazy-load anything visible above the fold.

Does this disable WordPress 7.0's built-in AI?

It can. WordPress 7.0 adds a core AI Client, Abilities API and Connectors screen, which stay dormant until you connect a provider. The Core tab's "Disable AI Features" toggle switches the whole subsystem off using core's own kill switch. It has no effect on WordPress 6.x.

I restored an orphaned image but the file is still missing.

Restoring reinstates only the database record. The file bytes are physically deleted when an attachment is staged for deletion, so a restored item must be re-uploaded if you need the actual file. This is why reviewing the candidate list before deleting matters.

Where are the settings?

Click **MBR Performance** in the WordPress admin toolbar at the top of the screen. Hover it to jump straight to any tab — every tab is listed, in the same order as on the settings screen — or to purge the page cache and flush OPcache.

Can MBR Performance switch on or tune OPcache?

No. OPcache is configured in `php.ini` by your host, and no plugin can enable it or change its memory. What the plugin can do is report its status on the Diagnostics tab, flush it, and make sure the files it writes itself take effect immediately. If the panel reports OPcache off or short of memory, pass the setting it names to your host.

I changed a PHP file but the site still runs the old code.

Your server probably has `validate_timestamps` switched off, which the OPcache panel on the Diagnostics tab will confirm. PHP then keeps running the compiled copy until OPcache is flushed. Use **Flush OPcache** on that panel or in the admin toolbar. Updates made through WordPress are refreshed automatically.

Mode B says "Still learning" on my front page and never moves on.

Update to 2.1.4 or later. Mode B counts distinct URLs, not visits, and a static front page only ever has one, so before 2.1.4 it sat at 1 of 3 (or whatever you had set) however much traffic it got. It was already being served optimised from the second visit — only the status was wrong. From 2.1.4 a static front page, the cart and the checkout need a single sample and show as "Learned — serving". There is nothing to clear.

I see "Deprecated: strip_tags(): Passing null..." at the top of the settings screen.

Update to 2.1.5 or later. The settings screen is a hidden page with no place in the admin sidebar, so WordPress could not work out its title and passed an empty value on, which PHP 8.1 and newer report as deprecated when debugging is switched on. It was harmless — nothing on the screen was affected — but 2.1.5 supplies the title itself, so the notice is gone and the browser tab reads "MBR Performance".

20 Glossary

Term	Meaning
AVIF	A modern image format, typically 20–30% smaller than WebP at equivalent perceived quality.
CLS (Cumulative Layout Shift)	A measure of how much page content unexpectedly jumps around while loading. Adding image width/height attributes reduces it.
Defer / Async	Two ways to stop a script blocking page render. Defer runs scripts in order after the page parses; async runs each as soon as it loads.
Delay	Holding non-critical scripts until the visitor first interacts with the page, so they don't slow the initial load.
Fetch Priority	A browser hint marking certain resources (such as the hero image) as high priority so they load sooner.
Field data	Measurements taken from real visitors, as opposed to synthetic measurements taken by a testing tool. See Real User Monitoring.
Import map	A small block of JSON WordPress prints so the browser knows how to resolve a script module's imports. It must appear before any module that depends on it, and combining modules would break it.
INP (Interaction to Next Paint)	How quickly a page responds when someone taps, clicks or types. It only exists once a real person interacts, so no synthetic test can measure it.
Interactivity API	The WordPress system that powers interactive blocks, built on script modules.
LCP (Largest Contentful Paint)	How long the largest visible element takes to appear — a key perceived-speed metric. Preloading the hero image improves it.
Lazy loading	Deferring off-screen images and embeds until the visitor scrolls near them, reducing initial page weight.
Minification	Removing comments and unnecessary whitespace from CSS or JavaScript without changing what it does, so the file is smaller to download. Complements compression rather than replacing it.
modulepreload	A browser hint telling it to fetch a script module early. Only useful if it appears before the module itself — which is what preload hoisting is for.
p75	The 75th percentile: the experience of your slower quarter of visits. Core Web Vitals are judged on p75 rather than an average, so a few slow visits aren't hidden by many fast ones.

Term	Meaning
Page cache	Storing the finished HTML of a page on disk and serving it to later visitors instead of building it again. See section 21.
Real User Monitoring (RUM)	Collecting performance metrics from actual visitors as they browse, rather than from a test run. The only way to observe INP.
Render-blocking	Resources (usually CSS or JavaScript) the browser must process before it can paint the page. Reducing them speeds first paint.
Safelist	A list of class names Mode B keeps regardless of whether they appear on the page as delivered — the guard rail for anything JavaScript adds after load.
Script module	JavaScript loaded as an ES module. WordPress 6.5+ uses these for the Interactivity API; they defer by specification and are never combined or minified.
Speculative loading	Fetching or pre-rendering the likely next page in advance so the next click feels instant.
Synthetic measurement	A performance reading taken by a tool rendering the page itself — repeatable and useful, but not what real visitors experience.
Template	A class of page that shares a layout — the front page, single posts, pages, archives, shop, product. Mode B analyses CSS per template rather than per URL.
Transient	A temporary cached value stored in the WordPress database with an expiry. Expired ones accumulate and can be cleaned up.
Used CSS (Mode A)	Inlining only the CSS a page actually uses and loading the full stylesheets asynchronously behind it, removing render-blocking unused CSS safely.
Used CSS (Mode B)	The aggressive variant: analyses each template from several sampled URLs and removes the analysed stylesheets outright rather than deferring them.
WebP	A widely-supported next-gen image format, substantially smaller than JPEG/PNG at similar quality.

Work through the tabs at your own pace, change one thing at a time, and let the Doctor guide your priorities. Used that way, MBR Performance gives you precise, transparent control over your site's speed without the guesswork — and with RUM switched on, real evidence that it worked.

21 Full Page Cache

Every other tab in this guide makes the page WordPress builds cheaper to build or faster to deliver. This one stops WordPress building it at all. The finished HTML of a page is written to disk the first time somebody visits it, and everybody after that is handed the file. Off until you switch it on.

The saving is larger than any other single setting in the plugin, because it removes the whole cost rather than trimming it. A typical WordPress page takes somewhere between 200 and 800 milliseconds of PHP and database work before the first byte leaves the server. A cached page takes a few milliseconds, and on the fastest tier a fraction of one.

How pages are served

There are three ways the cache can answer a request. The plugin uses the fastest one your hosting allows and falls back automatically, so a locked-down server means a slower cache, never a broken site. The Cache tab tells you which tier is active.

Tier	What runs	Speed
Static (.htaccess)	Nothing. Apache or LiteSpeed reads the file off disk and sends it. PHP is never started.	Fastest. Optional — you install the rules from the Cache tab.
Drop-in (advanced-cache.php)	PHP starts, but WordPress does not. No database, no plugins, no theme.	A few milliseconds. The default, and works on every host including Nginx.
In-WordPress fallback	WordPress loads, then the cached file is sent before the theme or the main query run.	Slower than the other two, but still a large improvement. Used where wp-content or wp-config.php is read-only.

Switching it on

- Go to MBR Performance → Cache and tick **Enable Page Caching**, then save.
- Saving installs the advanced-cache.php drop-in and adds the WP_CACHE line to wp-config.php for you. Both are removed again when you switch caching off or deactivate the plugin.
- Check the **Cache Status** panel at the top of the tab. Every row should show a tick. Anything that does not comes with a button to fix it.
- Visit a page of your site twice in a private browsing window. The second visit is being served from cache.
- Optionally, install the .htaccess rules from the Static Serving section to move to the fastest tier. Your existing .htaccess is backed up first.

NOTE**Your wp-config.php is backed up**

The first time the plugin edits wp-config.php it copies the original to wp-config.php.mbrpe-backup alongside it, and the edited version is syntax-checked before it is written — if it does not parse, the original is left exactly as it was. The same applies to .htaccess, which is backed up to .htaccess.mbrpe-backup. If your host makes wp-config.php read-only, as some managed hosts do, the Cache tab says so and shows you the one line to paste in by hand. Until you do, caching still works on the in-WordPress tier.

The settings

Setting	What it does	When to change it
Cache Lifetime	How long an entry may live before it is discarded. The default, Until purged, means entries live until something changes them.	Leave on Until purged unless part of your site changes without WordPress knowing — an external feed, a live stock count.
Pre-compress Pages	Writes a gzipped copy beside each page so the server never compresses the same bytes twice.	Leave on. Costs a little disk, saves CPU on every hit.
Separate Mobile Cache	Keeps a second copy of each page for phones.	Only if your theme genuinely sends different HTML to phones. A responsive theme does not.
Cache 404 Pages	Stores your not-found page so bots hitting dead URLs are cheap to serve.	Useful once your links are in order. Leave off while you are still fixing them.
Cache Feeds	Caches RSS and Atom output as well as HTML.	Safe on most sites. Leave off if a feed is generated per-subscriber.
Purge Everything on Edit	Clears the whole cache after any edit, instead of just the affected pages.	Only if your theme cross-links so heavily that targeted purging misses something.
Send Debug Headers	Adds X-MBR-Cache to every response, saying whether the page was cached and why not.	Leave on. It costs nothing and turns an afternoon of guessing into a minute of diagnosis.
Add HTML Signature	Appends a comment recording when the page was cached, visible in view-source.	Handy while testing. Seeing it proves the page came from cache.

What is never cached

A page cache goes wrong in exactly one way that matters: showing one visitor something meant for another. The following are excluded before anything else is considered, and none of them can be switched off.

- **Anyone logged in.** Signed-in visitors always get a freshly built page.

- **Any page that sets a cookie.** If the response carries a Set-Cookie header it is personalised by definition, so it is served but never stored.
- Any request carrying a login, comment-author, password or shopping-cart cookie.
- Password-protected posts, previews, the customizer, search results and page-builder editors.
- The WooCommerce cart, checkout and account pages — found by their real permalinks, so a shop whose checkout lives somewhere unusual is still protected.
- Anything a theme or plugin marks with the `DONOTCACHEPAGE` constant.
- Anything that did not return a clean 200, or whose HTML looks truncated. A page cut short by a fatal error is never frozen into the cache.
- A render made while Used CSS is still generating for that page, or Mode B is still learning its template. These are served but not stored, so the optimised page is what gets cached.

You can add your own exclusions on the Cache tab — by URL, by cookie name or by user agent. One per line. A URL rule ending in (*) matches everything beneath that path.

TIP**Do not exclude search engines**

It is a common instinct to add Googlebot to the user-agent exclusions, and it is almost always the wrong move. Crawlers are the visitors a cache serves most cheaply, they hit far more URLs than humans do, and Google explicitly wants to see the same page a visitor gets.

Query strings and campaign links

By default a URL carrying a query string is served live rather than from cache, because the plugin cannot know whether the parameter changes the page. There is one important exception, and it is switched on for you.

Around twenty analytics and advertising parameters — `utm_source` and its relatives, `fbclid`, `gclid`, `msclkid`, `mc_cid` and so on — are stripped before the cache is consulted. A page shared through a newsletter or an ad campaign therefore serves the same entry as its clean URL. Add any others your tools use in **Ignored Parameters**.

If you genuinely have parameters that change the page — a filtered archive, a sort order — tick **Cache Other Query Strings** and list them under **Cacheable Parameters**. Each distinct combination then gets its own entry, so list them explicitly rather than leaving the box empty.

Purging

A cache is only as good as its invalidation, so the plugin errs towards clearing a little too much rather than a little too little. Most of this happens without you.

When this happens	What gets cleared
You publish or edit a post	That post, the front page, the blog index, every category and tag it belongs to, its post type archive, its author archive, its year, month and day archives, and the posts either side of it — including the first twenty paginated pages of each archive.
Somebody comments	That post, and its comment feed if feed caching is on.

When this happens	What gets cleared
A category or tag is edited	That term's archive and its pagination.
A user profile changes	That author's archive and the front page.
WooCommerce stock moves	The product page and every archive it appears in.
Used CSS or Mode B finishes a page	That page, so its optimised version is what gets cached next.
Menus, widgets, theme, customizer, permalinks, plugins or updates change	Everything, because those change every page on the site.
You save or reset any settings tab	Everything, since the settings may change how pages are built and which combined or minified files they link to.
You clear the CSS or JS file cache	Everything, so no stored page links to a file that has just been removed.

To clear the cache by hand, use **Purge Entire Cache** on the tab, or the MBR Performance menu in the admin toolbar, which also offers **Purge This Page** while you are looking at a front-end URL.

Reading the debug header

With Send Debug Headers on, every response tells you what the cache did. Open your browser's developer tools, go to the Network tab, click the page request and look at the response headers. X-MBR-Cache will say one of the following.

Value	Meaning
HIT	Served from cache by the drop-in. X-MBR-Cache-Age tells you how old the entry is.
HIT-STATIC	Served by the web server from .htaccess rules, without starting PHP.
HIT-PHP	Served from inside WordPress, on the fallback tier.
HIT-304	The browser already had a current copy, so only headers were sent.
MISS	Not in the cache. The page was built normally and has now been stored.
BYPASS	This page is not cacheable. X-MBR-Cache-Reason names the rule — logged-in, excluded-cookie, excluded-uri, password-protected, search, preview, DONOTCACHEPAGE, used-css-generating and so on.

A BYPASS is usually correct rather than a fault. If you are logged in, every page will bypass — which is why testing in a private window matters.

Static serving

On Apache and LiteSpeed the Cache tab offers to add rewrite rules to your .htaccess so the server answers cache hits directly. The rules are deliberately cautious: anything with a query string, a login cookie, an unusual character or a non-GET method is handed back to PHP, which knows the full picture. The practical effect is that ordinary page views never touch PHP at all, which matters most on shared hosting where PHP worker slots run out before anything else does.

The rules are placed above WordPress's own block, because that block ends by handing every request to index.php and nothing after it would ever run. Removing them from the tab leaves the rest of your .htaccess untouched. Nginx has no .htaccess; if your site runs on Nginx the tab shows an equivalent snippet for your server block instead. Without it the drop-in still does the job perfectly well.

Proxies, CDNs and extra domains

The cache stores a separate copy of each page per scheme and per hostname, so a page requested over HTTPS is never answered with one stored over HTTP, and a site answering to two domains keeps them apart.

The scheme is normally taken from your site address, which is correct almost everywhere. If your site sits behind something that terminates HTTPS and forwards the request over plain HTTP — Cloudflare, a load balancer, some managed hosting — and you need the cache to follow the forwarded protocol, set the `trust_proxy` flag in the cache configuration. Leave it alone otherwise: a forwarded header can be set by anyone, so it is only trustworthy when you know a proxy is actually in front of you.

The hostname is checked against the addresses your site genuinely answers to. If you serve the same site on additional domains or aliases, add them with the `mbrpe_cache_hosts` filter, or requests to those domains will simply bypass the cache rather than being stored.

WARNING

One page cache at a time

If WP Rocket, LiteSpeed Cache, W3 Total Cache, WP Super Cache, Cache Enabler, WP Fastest Cache, FlyingPress, Surge, Breeze, Swift Performance or SiteGround Optimizer's file cache is active, MBR Performance will not start caching, and the Cache tab names the plugin responsible. Two page caches on one site produce stale and mismatched content that neither plugin can account for. Every other optimisation in this plugin carries on working normally alongside them.

If something looks wrong

- **A change is not showing** — purge the cache from the toolbar and reload. If that fixes it, tell us what you changed; it may be an event that should trigger a purge and does not.
- **Nothing is being cached** — check the Cache Status panel first. Then load a page in a private window and read X-MBR-Cache-Reason, which will name the rule responsible.
- **A form or a personalised block shows the wrong thing** — exclude that page by URL, or have the plugin that renders it define `DONOTCACHEPAGE`. If it sets a cookie it is already excluded.
- **You need to turn caching off in a hurry** — add the line below to wp-config.php. That bypasses the cache immediately without touching settings or files.

```
define( 'MBRPE_CACHE_DISABLE', true );
```

- **Something is badly broken** — deactivate the plugin. Deactivation removes the drop-in, the rewrite rules and every cached file, returning the site to exactly how it served pages before.